U208: eliminating a state using entry context

Comparing six proposed machines for Con(PA)
and constructing an explicit 208-state candidate

Research prepared for Vladimir Reshetnikov

29 October 2026

Abstract

The six U209 archives in the supplied ProveIt snapshot describe only two machines up to rooted, symbol- and direction-preserving state renaming. Four executable packages and the PDF-only package belong to one class; the Compatibility package belongs to the other. All executable versions preserve the same 3,909-primitive program and 5,083-arrow control graph.

We construct a deterministic, binary, one-tape machine with **208 working states and 416 transitions**, including blank initialization. Starting with the Joint U209 table, delete native state 172, redirect its incoming edge to 211, and replace the zero transition of 211 by a left move to 217. The justification is a conditional invariant missed by the earlier single-predecessor-tag searches: every reachable old 211 configuration reading zero has a one immediately to its left. Every old 172 entry has the two preceding bits 10. Four exact local identities then yield a positive block simulation, with one, three, or seven new steps per old step. The entire tape and head position agree at synchronization points.

The new theorem proves halting equivalence from state 0 on every common initial tape. It is directly formalizable in PA for numerically coded finite computations. Composing with the retained U209 logical calibration gives the conventional mathematical endpoint $\mathrm{PA} \vdash \neg\,\mathrm{Halt}(M_{208}, \mathrm{blank}) \leftrightarrow \mathrm{Con(PA)}$. Fresh symbolic replay verifies all 5,084 primitive contracts and all 416 literal rows. The complete startup trace is unchanged. The archive contains the exact tables, a deterministic builder, an independent local checker, comparison witnesses, reproducible discovery, and the retained parent. No new end-to-end proof-assistant certificate is claimed.

Principal executable: `machine/u208.tm`. Working states $0, \ldots, 207$; start 0; separate halt label 208; alphabet $\{0, 1\}$; all-zero two-way infinite tape; head at coordinate 0; only L/R moves.
Dense table SHA-256
`a2fee9e41311c0d43aaac871ea411be24eada17e6b3633c55f40b437101144e1`

Contents

1 Result, scope, and evidence **3**

2 Comparison of the six incoming proposals **3**

3 The exact 208-state construction **5**

4 Complete proof of the new reduction **6**

5 Discovery method and why the earlier screen missed it **9**

6 The logical calibration retained from U209 **10**

7 Verification of the delivered principal candidate **12**

8 A second 208-state table from Compatibility **13**

9 Limits, excluded shortcuts, and further work **13**

10 Archive guide and reproduction **14**

A Complete dense transition table **16**

B Source identities and evidence provenance **18**

1 Result, scope, and evidence

The requested threshold, fewer than 209 working states, is achieved by the principal 208-state candidate. Its complete dense table is printed in Appendix A. A second 208-state table, obtained from the distinct Compatibility parent, is supplied as a supplementary construction.

Three claims should be distinguished.

1. **New preservation theorem.** The principal U208 table and the exact Joint U209 table have the same halting behavior from state 0 on every common binary tape. Section 4 gives the complete proof from their literal transitions. It does not depend on the theory U or on the meaning of the source program.
2. **Direct implementation check.** Fresh symbolic induction checks the principal U208 table against every primitive of the unchanged source, for arbitrary natural register values and arbitrary allowed spare-register lists. Exact startup replay supplies the blank-input bridge. These checks corroborate the new preservation theorem and its implementation.
3. **Consistency endpoint.** The supplied U209 proof-search and interpretation arguments identify blank halting with a finite PA refutation. The new preservation theorem transfers that endpoint. Those substantial inherited mathematical arguments are retained and audited, rather than silently replaced by a K215 theorem about a different theory and table.

Theorem 1.1 (Relative arithmetic endpoint)**.** *For the exact principal tables delivered here,*

$$\mathrm{PA} \vdash \mathrm{Halt}(M_{208}, \mathrm{blank}) \leftrightarrow \mathrm{Halt}(M_{209}, \mathrm{blank}).$$

The proof uses fixed predecessor inventories, four constant-length literal traces, and induction on finite computations.

Corollary 1.2 (Endpoint after the retained calibration)**.** *Composing Theorem 1.1 with the mathematical calibration in the supplied Joint U209 ancestry gives*

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(M_{208}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(U) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

This packet supplies an ordinary mathematical proof and executable certificates. It supplies no newly compiled Lean, Rocq, MM0, or first-order PA endpoint proof. It does not assert Con(PA), unconditional nonhalting, or global minimality. The arithmetic equivalence is the statement to be proved; a consistency assumption is not used to prove that equivalence.

2 Comparison of the six incoming proposals

2.1 A fixed source snapshot

All six archives and the unary-theory archive [1] were retrieved from `VladimirReshetnikov/ProveIt`, directory `docs/incoming`, at commit

`343da82d7384d10ec9fd1c17361fc923b2d1d117`.

The source listing, exact URLs, archive sizes, Git blob identities, and SHA-256 hashes are recorded in `baseline/source_manifest.json`. Downloaded bytes were checked against the listed Git blob hashes. The comparison uses actual transition rows, not titles or declared state counts.

Short name	Archive stem following `U209_ConPA`	Class	Dense hash prefix
Plain	`_2026-10-29`	A	`65ccb606c8e970ad`
Compatibility	`_Compatibility_2026-10-29`	B	`bda91bd40880f96e`
Compatible Folds	`_Compatible_Folds_2026-10-29`	A	`5ded628d692ff442`
Composed	`_Composed_2026-10-29`	A	`3a69998d818b6e63`
Joint	`_Joint_2026-10-29`	A	`5ded628d692ff442`
Joint (1)	`_Joint_2026-10-29(1)` (PDF only)	A	`65ccb606c8e970ad`

Table 1: All six archives have the extension `.zip`. There are two rooted transition-table isomorphism classes. Full hashes are in the receipts.

Each dense table has working states 0 through 208, separate halt 209, and 418 transitions. All 209 working states are reachable in the directed transition graph from 0. This graph statement does not say that each state is reached in the particular blank-tape computation.

2.2 Exact equality and state-renaming witnesses

A rooted deterministic binary transition table has a canonical breadth-first enumeration: start with state 0, visit the read-0 edge before the read-1 edge, and assign a new canonical label on first encounter. Retain write bits, directions, and the distinguished halt. Equality of the resulting canonical tables is equivalent to a root- and halt-preserving state bijection because all working states are graph-reachable. The standalone comparison program also constructs the forced bijection directly and checks all 418 rows. For unequal classes it records an explicit conflict in that forced map. Thus the result is stronger than a comparison of file hashes.

Compatible Folds and Joint have *byte-identical* native and dense tables. Plain maps to Joint by the native-label cycles

$$(217\ 129\ 214), \qquad (215\ 204\ 216),$$

fixing every other label. Here the first cycle means $217 \mapsto 129 \mapsto 214 \mapsto 217$. Composed maps to Joint by

$$(217\ 129\ 204\ 214\ 216),$$

again fixing all other labels. These are verified transition-by-transition. No reflection of the tape, exchange of symbols, or altered start state is used.

The sixth archive contains only one PDF. We extracted all 418 transition rows from its appendix and reconstructed exactly Plain's dense file, including its hash. Its machine identity therefore does not rely on the hash printed in the prose. It is an alternative exposition of class A, not a third executable construction.

2.3 The common source program

The five executable packages have the same ordered instruction word:

$$\#\mathrm{CI} = 1558, \quad \#\mathrm{CD} = 1175, \quad \#\mathrm{CB} = 80, \quad \#\mathrm{CC} = 1095, \quad \#\mathrm{CH} = 1.$$

The total is 3,909 primitive sites. Resolving both outcomes of each conditional decrement gives 5,083 nonhalt control arrows. Two independently implemented resolvers, one based on subtree intervals and one on binary addresses and ancestor cuts, agree on the full ordinal control graph.

All class A source files are identical. They have maximum address depth 42; Compatibility's different association has maximum depth 41. Both have 197 distinct syntactic subtrees and maximum consecutive-zero run 6. The depth difference is a layout fact; it is not a proved asymptotic speedup. All five executable packages retain the same unary-theory ZIP, byte-for-byte. The proposals therefore concern decoder organization and physical state sharing, not different calibrating theories.

2.4 What each presentation contributes

All five full packages combine the former U211 savings associated with native states 180, 181, and 218. These savings cannot be added a second time by combining the current reports.

Plain presents thirty closed-factor rotations from U212, an arithmetic-entry repair, and three simultaneous deletions. Its useful additional research is a logical obstruction to deleting the dominated propositional row; see Section 9.

Composed obtains a renaming of Plain's machine and gives a clear account of shared helper entries and of the reverse direction of halting simulation. It also directly retains recovered U222 calibration material.

Compatible Folds uses seventeen rotations, an eleven-factor reassociation of the emitter, and three outer rotations. Its final machine is exactly Joint's, despite a different construction history.

Joint gives a compact simultaneous layout of eighteen selectors over a closed 3,096-primitive region, then performs the three deletions. Its deterministic builder, direct local checker, immutable engine binding, nonmutating replay, and complete nested ancestry make it the most convenient baseline for the present literal-table reduction. That choice is about auditability and packaging, not a smaller state count than its isomorphic peers.

PDF-only Joint (1) gives twenty-four reassociations from Contextual U211, corresponding to thirty when the parent's six are included. Its transition appendix is useful, but the archive itself contains no runnable builder or verification suite.

Compatibility is the sole different machine. It preserves a backward call through helper 129 by keeping the rows that call can actually read, while changing an unused branch, 204/read-1. An eight-factor reassociation then prepares the fold $180 \mapsto 216$. This is a useful example of preserving the relevant behavior of a shared helper without freezing its entire table. It is an alternative way to combine the same three savings, not a fourth independent deletion.

2.5 Fresh replay of both classes

One complete release from each class was freshly replayed. Joint's replay reconstructed its sealed machine and source, checked its local identities, and rederived all universal primitive contracts. Compatibility's replay likewise reconstructed both retained stages and rederived its final contracts, with fresh case records equal to its saved records.

3 The exact 208-state construction

The parent M_{209} is Joint's sparse native table, with SHA-256

`d0a155d3c683789f3bf5327c0cfb2a0cfd78076af8f1654d4ca426ccbb604d16`.

Check	Joint	Compatibility
Working states	209	209
Universal primitive contracts	5,084	5,084
Literal rows covered	418/418	418/418
Symbolic inference applications	1,250,049	1,248,748
Exact blank startup steps	48,140	48,140
Full release replay	PASS	PASS

Table 2: Fresh verification of the two distinct input designs. These are executable mathematical inductions in inherited Python engines.

Native labels are historical names. In particular native 211 is a working state, while native 254 is halt. They are not counts.

Define the projection

$$\rho(172) = 211, \qquad \rho(q) = q \quad (q \neq 172).$$

Delete both rows of native 172. Redirect every target 172 to 211; there is exactly one such incoming row. Finally change the read-zero row of 211. The complete difference in retained rows is

State	Read	Old action	New action
116	0	$(0, \mathrm{R}, 172)$	$(0, \mathrm{R}, 211)$
211	0	$(0, \mathrm{R}, 157)$	$(0, \mathrm{L}, 217)$

No other retained transition is edited. In particular, $\delta(211, 1) = (1, \mathrm{R}, 184)$ remains unchanged. The source tree, primitive order, operands, source addresses, header convention, and register program are all unchanged.

The remaining native states are sorted and assigned dense labels 0 through 207; native halt 254 becomes dense halt 208. `machine/state_map.json` records the complete bijection. The deterministic builder checks target closure, binary completeness, all 208 graph-reachable working states, and both expected file hashes.

4 Complete proof of the new reduction

4.1 Configurations and the relation

A configuration is (q, h, t), where q is a control state, $h \in \mathbb{Z}$, and $t : \mathbb{Z} \to \{0, 1\}$ is the tape. An action (w, D, z) writes w, moves by -1 or $+1$, and enters z. Reaching the distinguished halt ends the computation. The relation at simulation boundaries is

$$(q, h, t) \mathrel{\mathcal{R}} (\rho(q), h, t).$$

The old state is retained in this relation. In particular old 172 and old 211 have the same new representative but different entry guarantees.

4.2 Complete predecessor inventories

The following are the entire incoming inventories in M_{209}, checked against every one of its 418 rows. The notation $p/b : wDz$ records the source state and read symbol as well as the action.

$$\begin{aligned}
&\mathrm{In}(172) = \{116/0 : 0R172\},\\
&\mathrm{In}(116) = \{215/1 : 1R116\},\\
&\mathrm{In}(211) = \{163/0 : 0L211,\ 163/1 : 1R211\},\\
&\mathrm{In}(163) = \{132/1 : 1R163,\ 199/1 : 1R163,\ 217/0 : 0R163\},\\
&\mathrm{In}(217) = \{216/1 : 1R217\}.
\end{aligned}$$

None of 172, 116, 211, 163, or 217 is the initial state 0. Consequently every occurrence needed in the following backward arguments has the stated predecessor; there are no initial-time exceptions.

Lemma 4.1 (Two bits before state 172)**.** *In every computation of M_{209} beginning in state 0, on any initial tape, an occurrence of state 172 at head h satisfies $t(h-2) = 1$ and $t(h-1) = 0$.*

Proof. Its last transition must be 116/read-0, which writes zero at $h-1$ and moves right. The preceding transition into 116 must be 215/read-1, which writes one at $h-2$ and moves right. The final transition does not revisit $h-2$. Both facts therefore hold at entry to 172. □

Lemma 4.2 (A conditional bit before state 211)**.** *In every such old computation, an occurrence of state 211 at head h with $t(h) = 0$ satisfies $t(h-1) = 1$.*

Proof. If the last transition is 163/read-1, it has just written one at $h-1$ and moved right, proving the conclusion.

Otherwise the last transition is 163/read-0 from head $h+1$, moving left to h. Consider the preceding entry to 163. All three incoming edges to 163 move right from h. Entries through 132/read-1 or 199/read-1 would have written one at h, and the following move from $h+1$ would not change it. Both are incompatible with the present read-zero assumption. Thus the entry to 163 must have been 217/read-0. The sole entry to that occurrence of 217 was 216/read-1, which wrote one at $h-1$. The next two transitions touch h and $h+1$, so the one at $h-1$ persists. □

The second lemma is conditional on the current read symbol. The unguarded incoming inventory of 211 contains both a right arrival and a left arrival; it does not supply a uniform immediate-left tag. This distinction is the reason the stronger invariant helps.

4.3 Four whole-tape identities

Set the old head to coordinate zero by translation. The following table lists all exceptional old-state/read cases. The stated cells are known; every other cell is arbitrary and remains unread by the replacement block.

For old 211/read-0, Lemma 4.2 gives the tape fragment 10 at coordinates $(-1, 0)$. The new path is

$$211@0 \longrightarrow 217@{-1} \longrightarrow 215@0 \longrightarrow 157@1.$$

The successive rows are new 211/read-0, unchanged 217/read-1, and unchanged 215/read-0. All writes preserve the scanned bit. This is the old $211/0 = (0, \mathrm{R}, 157)$ action after three steps.

Old state	Read	Needed context	New steps	Final state/head
172	0	$t(-2) = 1,\ t(-1) = 0$	7	$67, -1$
172	1	none	1	$184, +1$
211	0	$t(-1) = 1$	3	$157, +1$
211	1	none	1	$184, +1$

Table 3: Each endpoint is exactly the old one-step endpoint after applying ρ. Both machines preserve the complete tape in these cases.

Step	State	Head	Read	Executed action	Next state	Next head
1	211	0	0	$(0, \mathrm{L}, 217)$	217	-1
2	217	-1	0	$(0, \mathrm{R}, 163)$	163	0
3	163	0	0	$(0, \mathrm{L}, 211)$	211	-1
4	211	-1	0	$(0, \mathrm{L}, 217)$	217	-2
5	217	-2	1	$(1, \mathrm{R}, 215)$	215	-1
6	215	-1	0	$(0, \mathrm{R}, 157)$	157	0
7	157	0	0	$(0, \mathrm{L}, 67)$	67	-1

Table 4: The seven-step simulation of old 172/read-0. The tape is 100 on the three displayed cells throughout; its arbitrary exterior is untouched.

For old 172/read-0, the fragment at $(-2, -1, 0)$ is 100. The complete new trace is displayed explicitly; no old version of a changed helper is used during this trace.

The old action $172/0 = (0, \mathrm{L}, 67)$ has exactly that endpoint. The common read-one action $(1, \mathrm{R}, 184)$ handles the remaining two cases in a single step. None of the four blocks enters halt internally.

For every old row outside states 172 and 211, the new action writes the same bit, moves in the same direction, and enters the ρ-image of the old destination. These 414 ordinary rows therefore simulate in one step.

4.4 Simulation, reflection of halting, and the clock

Theorem 4.3 (Uniform preservation of halting)**.** *From state 0, the exact principal U209 and U208 tables halt on exactly the same initial binary tapes. At matching points their tapes and head positions agree. If the old computation halts after n steps, the new one halts after*

$$\tau(n) = n + 6A(n) + 2B(n),$$

where $A(n)$ counts old 172/read-0 transitions and $B(n)$ counts old 211/read-0 transitions among those n steps. In particular, $n \le \tau(n) \le 7n$.

Proof. The initial configurations satisfy $\mathcal{R}$, because $\rho(0) = 0$. Induct on the number of old steps. In an ordinary state, the single new step commutes with ρ. In states 172 and 211, the old-machine lemmas supply the relevant guards, and the appropriate block in Table 3 returns to $\mathcal{R}$. Summing the block lengths gives the stated clock formula. All block lengths are positive.

If the old computation halts, the constructed finite new computation reaches the corresponding halt. Conversely suppose the new machine halts at time T. If the old machine did not halt in its first

T steps, the induction would construct a new computation through $\tau(T) \geq T$ whose synchronized endpoints are working states and whose internal blocks contain no halt. This contradicts halting at time T. Hence the old computation also halts. □

There is no circularity in the internal revisits to new 211 and new 163. The lemmas concern only the old machine at old-step boundaries. The seven-step new trace is executed directly, including the new 211/read-0 row on its second use. We do not assert the old invariant at each transient configuration of the new machine.

For the PA version, code finite-support initial tapes and finite computation histories by natural numbers. The predecessor facts and local traces are fixed finite arithmetic verifications. Induction defines the synchronization clock and proves the same endpoint relation; the finite T-step reflection argument proves the reverse implication. Thus the blank instance is an ordinary PA derivation in principle, without quantifying over arbitrary sets of integers inside PA. The external all-tape theorem is stronger in scope than the particular arithmetic blank-input statement required here.

5 Discovery method and why the earlier screen missed it

The existing Joint research programs examine direct aliases and folds supported by immediate incoming-edge tags. The published negative runs are bounded screens of restricted templates, not proofs of minimality. The two states used here agree on read-one but differ on read-zero:

	0	1
172	$0L67$	$1R184$
211	$0R157$	$1R184$.

Immediate tags describe a zero to the right after a backward arrival at 211; they do not determine the bit needed by a new backward selector. Looking a few transitions further back, while conditioning on reading zero now, rules out two incoming branches and forces the missing one.

The included discovery program constructs finite backward symbolic histories from the exact old table. It uses incoming write/read consistency to discard impossible histories and retain forced bits. Candidate replacement rows are screened on concrete examples to reject obvious failures, then accepted only by exhaustive symbolic execution of every remaining guarded case with arbitrary exterior. Random screening is used only for rejection; it is not a correctness certificate.

The cleaned discovery script, with history depth 3 and macro bound 7, rediscovers the two symmetric orientations of this fold. The final proof was then simplified to Lemmas 4.1 and 4.2; the release checker does not trust the discovery engine, its random screen, or its general backward-history implementation. It reads the old and new tables and checks the short, fixed premises independently.

The invariant is necessary for this argument. For example, at an arbitrary configuration of old 211 with local fragment 100 at coordinates $(-2, -1, 0)$, old 211/read-0 goes to 157@1; the new control follows the seven-step behavior ending at 67@−1. Such a configuration is excluded by Lemma 4.2 on old runs from state 0. The result is therefore a reachability-sensitive simulation, not equality of the two state functions on unrestricted configurations.

6 The logical calibration retained from U209

6.1 The unary theory and its restriction

The supplied unary manuscript [1] uses classical equality-free first-order logic on nonempty domains, with one binary predicate E. Write

$$\mathrm{Sub}(x,a) := \forall z\,(E(z,x) \to E(z,a)).$$

For each formula φ with free variables contained in $\{x\}$, the theory U contains

$$\forall a\,\exists b\,[\forall x\,(E(x,b) \to \varphi(x)) \;\wedge\; \forall x\,(\mathrm{Sub}(x,a) \to (\varphi(x) \to E(x,b)))]\,. \tag{U}$$

The source parameter a is not allowed free in φ. There is no bound on bound-variable count, formula size, or quantifier complexity. No extensionality axiom is assumed. In particular, replacing $\varphi(x)$ by $\mathrm{Sub}(x,a) \wedge \psi(x)$ would violate the stated unary restriction, so ordinary unrestricted separation cannot simply be inserted into the lower interpretation.

6.2 What the manuscript proves

The manuscript contains a substantial ordinary proof of the U/PA consistency equivalence. Our review found no concrete defect in the following delicate points; a new formal endpoint was not produced.

Coextensionality identifies objects with the same membership profile. Numerical objects are saturated, transitive, linearly ordered in the relevant membership sense, and satisfy least/greatest endpoint conditions for represented intersections at initial cutoffs. These definitions do not assume external finiteness or external well-foundedness.

The lower construction first proves an existing nonempty numerical object has a greatest member that supplies its predecessor. It then proves induction *before* proving total successor. A unary cover selects failures inside a numerical source. If there is an internal failure, choose its least class; if there is none but the source itself fails, the source is the least failure. In either case its predecessor contradicts the induction step. This second case avoids circular reliance on an already available successor.

Parameters are internalized through a legal unary predicate of the form

$$Q(v) := \forall \vec{p}\,(\mathrm{BaseStep}(\theta,\vec{p}) \to \theta(v,\vec{p})).$$

Least-failure arguments then recover exact bounded comprehension with numerical parameters. A least counterexample tuple is bound inside the unary matrix; an independent support contains the selected objects as subsets. The proved invariance of the target formula in its numerical parameters makes equivalent minimal tuples select the same profile. This is a formula-specific syntactic construction, not a truth predicate for all formulas.

Total successor and exact powers of numerical sources follow. Saturated singleton and pair profiles provide relation coding modulo coextensionality. Finite graph histories define addition and multiplication. Old graph witnesses are existentially bound inside the construction; only numerical values remain free parameters. Pointwise uniqueness and numerical bounds are established before the relevant graph extensions. The translated induction schema gives a full interpretation of PA.

In the other direction, PA interprets $E(x,a)$ as the x-th binary digit of a. PA proves that $\mathrm{Sub}(x,a)$ entails $x \le a$. For any fixed translated formula, induction constructs its characteristic code on that bounded interval. This verifies even the unrestricted weak covering schema. The same interpretation

satisfies global irreflexivity. These arguments are available in PA models, rather than only in the standard natural numbers.

Uniform renaming, substitution, relativization, and finite derivation templates provide the advertised proof translations. Their arithmetic verification uses induction on formulas and finite derivations. The manuscript does not deliver an implemented arbitrary-proof translator or a checked Lean/Rocq/MM0 endpoint; that is an explicit formalization limit.

6.3 What the literal machine actually enumerates

The U209 lineage does not compile the unary ZIP's separate positive dyadic stream. It searches an inherited native numerical calculus for unrestricted weak covering W plus global irreflexivity IR. Its target is native code 1, and code 0 is a truth alias. The positive stream has a different target, code 0, and a different codec. A small constructor count in that stream cannot be read as a state bound for the literal U209 or U208 tables.

The consistency translations form the cycle

$$\mathrm{Con(PA)} \implies \mathrm{Con}(W + \mathrm{IR}) \implies \mathrm{Con}(U) \implies \mathrm{Con(PA)}.$$

The machine's additional native-calculus bridge is substantive. The inherited numerical constructors use a Cantor pairing P, implication $4P$, membership $4P + 1$, universal quantification $4P + 2$, and auxiliary equality $4P + 3$. The alias $I(0, 0) = 0$ and the target $R(0, 0) = 1$ require explicit treatment. Auxiliary equality is calibrated through membership indiscernibility rather than importing unavailable primitive equality rules.

The role-based covering emitter uses one integer both as matrix code and output label. Its numerical bounds ensure freshness for positive codes; the zero truth alias and source/output collision are handled separately. Padding recovers every required cover instance. We inspected the retained U222 native calibration and the later U214 role-emitter argument, rather than inferring these properties from the final state count.

6.4 Startup and proof-search completeness

Blank startup enters an interior axiom-generation suffix. Its existing cached value must not simply be treated as a theorem. The inherited argument produces nine theorem-valued heads above an opaque stack tail, then uses a diagonal credit bound to show the unsafe tail is never consumed. A ghost stack length is retained because the numerical pairing has a zero alias. Strong source compilation, including its raw-zero exceptions, gives cofinally many successful refutation candidates and termination of unsuccessful candidate evaluations. These are the source-to-native soundness and completeness dependencies behind the blank-input endpoint.

The exact Joint release and its full nested ancestry are retained as `baseline/U209_ConPA_Joint_2026-10-29.zip`. The unary-theory archive is also retained separately. Some historical Rocq files in that ancestry are reference excerpts with external imports; their presence is not a new runnable U208 formal verification project.

The K215 ZFC formalizations and the K230 account of Theory K are useful reference points [4, 3, 5, 2]. They establish different literal-table endpoints. The present seven-state numerical difference from 215 does not by itself compare the intrinsic complexity of PA and ZFC encodings.

7 Verification of the delivered principal candidate

7.1 Independent local certificate

`code/verify_fold.py` imports none of the discovery, source compiler, or universal-runtime modules. It pins the exact parent, reads the actual candidate, verifies the complete predecessor inventories, checks all ordinary transition correspondences, and executes the four exceptional blocks with an opaque exterior. A read outside the supplied finite context fails; equality of the final finite cells therefore proves equality of the full arbitrary tape.

Eight deliberately corrupted constructions are rejected, including wrong first and second entry bits, a wrong deeper predecessor, a wrong helper direction or write, and a wrong selector continuation. The local checker passes under ordinary Python and under `-O`; its required conditions use explicit exceptions rather than removable assertions. The discovery script is not invoked by the release verifier.

7.2 Unchanged source interfaces and startup

The source bytes are identical to Joint's. All 3,908 internal source prefixes were traversed against both tables, with endpoints commuting with ρ. The maximum prefix traversal length changes from 49 to 50 literal steps; no source address changes.

The complete 48,140-step blank startup was executed independently on both tables. The entire transition trace, final head, state, and finite support of the tape agree. No edited or deleted row is visited during that prefix. The endpoint is native state 84, head -2, source ordinal 136, address `001100110101010`, and logical registers

$$[1, 0, 13, 0, 0, 1, 0, 0, 1],$$

with 43 spare registers all equal to 1. The full canonical frame has 120 one-cells and zero exterior, recorded in the startup receipt. This finite check verifies initialization; it is not evidence of unbounded nonhalting by itself.

7.3 Fresh universal primitive replay

The inherited native-label induction engine was run on the final 208-state table. For each source primitive it checks the complete tape endpoint with arbitrary values of nine primary registers, an arbitrary natural gap, and an arbitrary nonempty finite spare-register list. Conditional decrement has separate zero and positive cases, giving $3909 + 1175 = 5084$ contracts.

The engine's inference rules are literal transition steps, induction over a unary scan length for a corresponding self-loop, and structural induction over a finite list after checking its one-block transformation. Four one-block schemas are rederived from the literal table. Durations are affine expressions whose coefficients are checked nonnegative and whose constant terms are positive. The minimum constant is 5.

The Python induction engine is inherited, readable, and explicitly part of the trusted executable evidence. It is not a proof-assistant kernel. The local proof in Section 4 transfers the parent's theorem independently of rerunning this engine; the direct replay is valuable because it checks the delivered table against the unchanged source as well.

Principal U208 check	Result
Working states / complete transition rows	208 / 416
Graph-reachable working states	208
Exact exceptional local cases	4
Other old rows commuting with projection	414
Rejected local corruptions	8
Source primitive sites / ordinal control arrows	3,909 / 5,083
Internal source-prefix interfaces	3,908
Universal primitive contracts	5,084
Symbolic inference applications	1,251,865
Literal transition rows covered	416 / 416
Reproved finite-list block schemas	4
Exact common startup steps	48,140

Table 5: Verification facts for the principal candidate. Symbolic contracts are unbounded mathematical identities, not 5,084 sampled register tuples.

8 A second 208-state table from Compatibility

The same new idea applies to the distinct Compatibility U209 table. Delete 172, redirect 116/read-0 to 211, and use

$$\delta(211, 0) = (0, \mathrm{L}, 214).$$

The relevant role correspondence is

	Folded predecessor	New backward helper	Return selector
Joint	163	217	215
Compatibility	216	214	163

In Compatibility, 172 is still entered through 116/read-0, now with 116 entered through 163/read-1. State 211 is entered from 216; a backward arrival reading zero forces the 214/read-0 predecessor, whose incoming 217/read-1 supplies the one. The same seven-, three-, and one-step identities follow with the listed labels.

`code/check_alternative.py` verifies this as an explicit label instantiation of the local theorem. This label correspondence is not a claim that the two complete parent tables are isomorphic. The alternative dense table is `machine/u208_compatibility.tm`, SHA-256

`9865dcd7bd44deece098c16dcec7b90a96cf111592cbd37e3531b50f7b09276c`.

Its direct local preservation theorem is checked; a separate fresh full primitive replay for this supplementary table was not performed. The principal table remains the one with the full new runtime and startup receipts.

9 Limits, excluded shortcuts, and further work

The specific advance over the supplied proposals is the conditional three-transition entry argument and the use of an already present decoder to distinguish two local contexts. The final modification

changes only two retained rows and removes one state. No new logical axiom system, proof-search scheduler, source reassociation, or register representation is required for this saving. Novelty is claimed relative to the supplied archives; no broader historical priority claim is made.

A larger search followed the same family of templates to history depth 8 and replacement-block bound 31. It found the same two orientations. A follow-up search on U208 examined 448 oriented row-compatible pairs and 369,152 replacement-row choices without another success; a direct alias search was also unsuccessful. These are restricted experimental negative results. They establish neither a lower bound of 208 nor exhaustion of possible representation changes.

Two tempting theory-level shortcuts are already ruled out by the input research. Deleting either freshness gate alone from the unchanged covering emitter has small-code counterexamples. Deleting the whole "dominated" propositional row is also invalid as a route to the same calibration: Plain constructs a Boolean valuation with implication interpreted as equality, membership false, auxiliary equality true, and quantifiers acting identically on truth values. All remaining axioms are designated and modus ponens/generalization preserve that property, while the target is not designated. PA can verify underivability of the target in the reduced calculus. If PA is consistent, that calculus cannot retain a PA-provable equivalence to Con(PA), by second incompleteness. These are inherited obstructions, not new discoveries here.

The most direct next verification task is to encode the two entry lemmas and the generic positive-block simulation theorem in a small formal framework, then instantiate them with the sealed table. That would formalize the new reduction alone. A fully checked Con(PA) endpoint still requires formalization of the inherited U/native/source calibration. For further state reduction, guarded history information is a useful additional search dimension; the present bounded searches do not decide what can be achieved with longer contexts or more global transformations.

10 Archive guide and reproduction

The principal deliverables are this PDF and its complete LaTeX source, the dense and native U208 tables, and deterministic offline verification.

Path	Contents
`report.tex`, `report.pdf`, `paper/`	Article, generated full table, and provenance appendix.
`machine/`	Principal and alternative literal tables; unchanged source; dense/native map.
`code/`	Builder, independent local checker, standalone comparison, source/startup audit, and universal runtime wrapper.
`evidence/`	Actual local, comparison, startup, and universal-contract receipts.
`comparison/`	Exact tables/source maps, notices, and reports from the six incoming packages.
`baseline/`	Complete sealed Joint parent ZIP, unary theory ZIP, exact parent tables, and source manifest.
`research/`	Clean discovery script, discovery receipt, and detailed comparison/theory audit notes.
`licenses/`, `NOTICE.txt`	Preserved attribution and inherited terms.
`SHA256SUMS`	Hashes of the other sealed release files.

Use Python 3.10 or later, with its standard library and assertions enabled:

```
python3 -B reproduce.py --output /tmp/u208-verification.json
```

The script verifies the manifest, reconstructs the exact machine bytes in a temporary directory, checks the independent local proof both normally and under `-O`, checks the alternative, compares all six snapshots, replays startup, and freshly rederives all 5,084 contracts. The sealed release is not modified. The runtime phase takes a few minutes on the preparation environment; elapsed time is not part of the certificate.

```
python3 -B reproduce.py --quick
```

explicitly skips the universal runtime phase and reports that omission. The optional `-parent-replay` also unpacks and replays the complete retained Joint release. If Poppler's `pdftotext` is present, comparison freshly extracts the PDF-only table. Otherwise it verifies the delivered extracted table and explicitly records that extraction was not rerun; the standalone comparison's `-require-pdf-extraction` makes that tool mandatory. Preparation included fresh extraction.

To rerun discovery separately:

```
python3 research/discover_folds.py baseline/u209.native.tm \
  --output /tmp/u208-discovery.json
```

Discovery is optional and is not a prerequisite for verifying the fixed construction. To rebuild the article in a working copy, run `pdflatex -interaction=nonstopmode -halt-on-error report.tex` twice. TeX versions and PDF metadata may change PDF bytes; the machine bytes are fixed and checked independently of typesetting.

A Complete dense transition table

This is the principal `machine/u208.tm`. Each entry wDz means write w, move D, and enter z. Start in 0 on the all-zero tape; 208 is the separate halt label. Two state groups are printed side by side to save space. Every row is generated directly from the sealed literal table, not transcribed manually.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R92	1R0	104	0R21	1R120
1	0L1	1L2	105	0R21	0L10
2	0L3	1L2	106	0R61	1R42
3	0L5	1L2	107	0R121	1R8
4	1R6	0L4	108	0R101	1R122
5	0L9	1L2	109	0L207	0L95
6	0R6	0R10	110	0R124	1R104
7	0R189	1R159	111	0R115	1L72
8	0R14	1R13	112	0R194	1R180
9	0L15	1L2	113	0R160	1R183
10	0L16	0R10	114	0R148	1R146
11	0L26	1R128	115	0R11	1R129
12	0R19	1R20	116	0L131	0L1
13	0L89	1L31	117	0R39	1R72
14	0L11	0R22	118	0R61	1R132
15	0L23	1L2	119	0R61	1R133
16	1L1	0L1	120	0R70	0L16
17	0R24	0R32	121	0R14	1R132
18	0R98	1R113	122	0R21	1R134
19	0R28	1R29	123	0R151	1R188
20	0R30	1R31	124	0R137	1L20
21	0R24	0R59	125	0R130	1R179
22	0R33	1R22	126	0R203	1R156
23	0L34	1L2	127	0R75	1R175
24	0L35	0R0	128	0R75	1R205
25	0R36	1R25	129	0R202	1R157
26	0R37	1R38	130	0R12	1R141
27	0R39	0L40	131	0R30	1R69
28	0R41	1R42	132	0R39	1L124
29	0R43	1R44	133	0R153	0L16
30	0R45	1R46	134	0R154	0L16
31	0R21	1R47	135	0R88	1R136
32	0R48	1R32	136	0R196	1R140
33	0R32	1R33	137	0R24	0R48
34	0L74	1L2	138	0R99	1R37
35	1R35	1L1	139	0R171	1R148
36	0R50	1R36	140	0R96	1R159
37	0R17	1R111	141	0R174	1R165
38	0R13	1R52	142	0R12	1R159
39	0L11	0R36	143	0R88	1L165
40	0L4	1L53	144	0R88	1R158
41	0R24	0R25	145	0L152	1L163
42	0R13	0L16	146	0R84	1R88
43	0R55	1R56	147	0R159	1R12
44	0R57	1R58	148	0R83	1R143
45	0R24	0R54	149	0R144	1R155
46	0R60	1R61	150	0L66	1L141
47	0R62	0L16	151	0R150	1R169
48	0R25	1R48	152	0R159	1R114
49	0L35	0R63	153	0R97	1R97

Continued on the next page

State	Read 0	Read 1	State	Read 0	Read 1
50	0R16	1R50	154	0R49	0R32
51	0R147	1R143	155	0R191	1R187
52	0R67	1R68	156	0L194	1R194
53	0L40	1L53	157	0L186	1R96
54	0R22	1R54	158	0R167	1R178
55	0L11	0R50	159	0R19	1R170
56	0R60	0L16	160	0R164	1R186
57	0R69	1R70	161	0R159	1R96
58	0R61	1R71	162	0R125	1R164
59	0R54	1R59	163	0R97	1R162
60	0R49	0R25	164	0L189	1R142
61	0L28	1L72	165	0R146	1R171
62	0L117	1L30	166	0R96	1R123
63	0R73	1R63	167	0R161	1R143
64	0R97	1R150	168	0R148	1R87
65	1R184	1L53	169	0R161	1R96
66	0R144	1R145	170	0R180	1R122
67	0R76	1R208	171	0L192	1R60
68	0L196	1R31	172	0R145	1R88
69	0R41	1R62	173	0R152	1R201
70	0L89	1R78	174	0R140	1R84
71	0R55	1L79	175	0R174	1R75
72	0R80	1R81	176	0R126	1R168
73	1R0	1R63	177	0R138	1R116
74	0R74	1R82	178	0R97	1R51
75	0L145	1L141	179	0R167	1R51
76	0R85	1R86	180	0R17	1L131
77	0R20	1R38	181	0R143	1R149
78	0R89	1L132	182	0R76	1R116
79	0L108	1R56	183	0R106	1R124
80	0R49	0R33	184	1L40	1L109
81	0R91	0L42	185	0R174	1R156
82	0R177	1R77	186	0R181	1R182
83	0R96	1R12	187	0R66	1R204
84	0R88	1R97	188	0R140	1R197
85	0R206	1R21	189	0R84	1R190
86	0R116	1R100	190	0R157	1R185
87	0R173	1R166	191	0R64	1R7
88	0R102	1R103	192	0R150	1R127
89	0R80	1R79	193	0R18	1R94
90	0R101	1R195	194	0L200	1R172
91	0R105	1R106	195	0R104	1R96
92	0R65	0R0	196	0R90	1R88
93	0R27	1R107	197	0R139	1R199
94	0R182	0L207	198	0R150	1R112
95	1L109	1L95	199	0R201	1R200
96	0R19	1R110	200	0R156	1R198
97	0R60	1R60	201	0L123	1R167
98	0R68	1R135	202	0R196	1R28
99	0R93	1R108	203	0R163	1R192
100	0R105	0L207	204	0R146	1R176
101	0R117	1R118	205	0R161	1R193
102	0R71	1R58	206	0R45	1R100
103	0R57	1R119	207	0L4	0L207

B Source identities and evidence provenance

Principal machine and unchanged source

u208.tm

a2fee9e41311c0d43aaac871ea411be24eada17e6b3633c55f40b437101144e1

u208.native.tm

b348a466a2c316511e0037ea3824de29d4c6ad37338404fbc98a3968fdb25135

sites.json

cddbed7f20e7a8b085b7ca0e76811d07b021522375643b5d1134eec0d2d23164

Retrieved incoming archives

The hashes below identify the complete original ZIP bytes, including nested artifacts.

Original Git blob hashes, sizes, and pinned download URLs accompany them in the JSON manifest.

U209_ConPA_2026-10-29.zip

7fd484091428643f211ea55d8750c127734705c0f91cd549b74fdc2c0d598f9f

U209_ConPA_Compatibility_2026-10-29.zip

35c9c1c9f5526f7f85200fd2b2f872ade9c05655b6722d40d9dc5232958a8ec4

U209_ConPA_Compatible_Folds_2026-10-29.zip

afee49ed265309f84e69eb0d92a5cfba929377381e570a2c562fa2f5702d6f4d

U209_ConPA_Composed_2026-10-29.zip

5ad6da480e4fcf22ad020659070f487d73efbd8aefe9a20fcf5f00195853b9d0

U209_ConPA_Joint_2026-10-29(1).zip

981218c643c6cc3a62afaa93e9c6cf0f73dce203e1b344d33ee59e9a9a6bd91e

U209_ConPA_Joint_2026-10-29.zip

9dd2199baffa0244da981384c81920afdccc353509c3c8745b28e95a93afc7a1

pa_unary_power_cover.zip

0fcb1117a5353766a3366bab04e56b4efbd0607833cdc73ffb34b67e6808b0ff

Retained executable evidence

The new local certificate is `evidence/independent_fold.json`; the optimized replay is `evidence/independent_fold_optimized.json`. The exact source/startup receipt is `evidence/source_startup.json`. Fresh initial universal data are summarized in `evidence/u208_runtime_initial.json`, with all case records in `evidence/u208_runtime_cases.json.gz`. The standalone six-way comparison is `evidence/six_way_comparison.json`; its PDF extraction was freshly executed. Original-parent full replays are recorded in `evidence/joint_replay.json` and `evidence/compatibility_replay.json`.

The new runtime wrapper rederives its case records and requires equality with the sealed records. These records are supporting executable evidence; the fixed local proof does not assume the search algorithm is sound. The archive manifest binds the code and machine bytes supplied for review.

References

[1] Vladimir Reshetnikov, ProveIt, six U209 archives and `pa_unary_power_cover.zip`, snapshot `343da82d7384d10ec9fd1c17361fc923b2d1d117`, 29 October 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/343da82d7384d10ec9fd1c17361fc923b2d1d117/docs/incoming. Exact filenames and archive hashes are listed above and in the source manifest.

[2] Jeff's busy-beaver repository, `ZF/m254`, branch `zf-254`. https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.

[3] ProveIt K215 verification: Rocq, Lean, and audit materials. https://github.com/VladimirReshetnikov/ProveIt/tree/343da82d7384d10ec9fd1c17361fc923b2d1d117/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[4] Elliot Glazer, `k215-pa`, the literal K215/Con(ZFC) theorem and MM0 verification receipts. https://github.com/elliotglazer/k215-pa.

[5] Elliot Glazer, `k230-rocq`, README, "The theory K." https://github.com/elliotglazer/k230-rocq/blob/main/README.md#the-theory-k.